Индивидуальное задание для практики 3-4.1

Класс Unit

```python
class Unit:
    def __init__(self, name: str, unit: str):
        self._name = name
        self._unit = unit
        self._coefficients = {}

    # Возвращает название с заглавной буквы
    def name(self):
        return self._name[0].capitalize() + self._name[1:]

    def symbol(self):
        return self._unit.lower()  # Возвращает обозначение в нижнем регистре

    def set(self, other_unit, coefficient):
        # Проверка что передаваемый класс принадлежит Unit
        if isinstance(other_unit, Unit):
            self._coefficients[other_unit.symbol()] = coefficient
        else:  # Иначе ошибка
            raise TypeError("other_unit must be Unit")

    def remove(self, other_unit):
        # Проверка что передаваемый класс принадлежит Unit
        if isinstance(other_unit, Unit):
            del self._coefficients[other_unit.symbol()]
        else:  # Иначе ошибка
            raise TypeError("other_unit must be Unit")

    def coefficients(self):
        # Возваращаем коэффициенты в виде кортежа
        return tuple(self._coefficients.items())

    def __iter__(self):
        # Итератор для tuple()
        return iter(self._coefficients.items())

    def __str__(self):  # Выводит описание в нужном формате
        return self._unit
```

Класс Measure

```python
class Measure:
    def __init__(self, unit: Unit, value):
        self._unit = unit
        self._value = value

    def value(self):
        return self._value

    def convert(self, unit: Unit):
        # Достаем коэффициент под нужный символ
        coefficient = self._unit._coefficients.get(unit.symbol())
        if coefficient == None:
            return None
        # Возвращаем новый объект с пересчитанным значением
        return Measure(unit, self._value * coefficient)

    def __add__(self, other_measure):
        try:
            curr = other_measure(self._unit)
            return Measure(self._unit, self.value() + curr.value())
        except Exception:
            return None

    def __call__(self, *args, **kwds):
        coefficient = self._unit._coefficients[args[0].symbol()]
        if coefficient == None:
            return None
        return Measure(args[0], self._value * coefficient)

    def __str__(self):  # Выводит описание в нужном формате
        return str(int(self._value)) + str(self._unit)
```

Оба класса лежат в solution.py

Класс Utin описывает систему измерения, а класс Measure само измерение, позволяет переводить в различные системы измерения и поддерживает сложение.

Результаты

```
from solution import Measure, Unit
```

```
meter = Unit("метр", "м")
kilometer = Unit("километр", "км")
meter.set(kilometer, 1 / 1000)
meters = Measure(meter, 10000)
print(meters.value())
kilometers = meters.convert(kilometer)
print(meters, kilometers)
kilometers = meters(kilometer)
print(meters, kilometers)
print(meters + kilometers)
kilometer.set(meter, 1000)
print(meters + kilometers)
```

```
10000
10000м 10км
10000м 10км
None
20000м
```

```
meter = Unit("метр", "м")
print(meter.name())
print(meter.symbol())
kilometer = Unit("километр", "км")
meter.set(kilometer, 1 / 1000)
print(meter.coefficients())
print(tuple(meter))
meter.remove(kilometer)
print(meter.coefficients())
print(tuple(meter))
print(meter)
```

```
Метр
м
(('км', 0.001),)
(('км', 0.001),)
()
()
м
```